CARPETA DE PROYECTO: PLATAFORMA WEB UNIGAMES

Carrera: Tecnicatura Universitaria en Programación
Universidad Tecnológica Nacional (UTN) - Facultad Regional Tucumán (FRT) - Anexo Concepción

Tutor Académico: Ing. Raúl E. Prado
Fecha de presentación: Diciembre 2026

Integrantes:

- [Nombre Integrante 1] - Legajo: [00000]
- [Nombre Integrante 2] - Legajo: [00000]
- [Nombre Integrante 3] - Legajo: [00000]

2. ÍNDICE

1. Portada
2. Índice
3. Resumen Ejecutivo
4. Introducción
5. Estado del Arte
6. Descripción del Proyecto
7. Metodología
8. Recursos Necesarios
9. Plan de Trabajo
10. Resultados Esperados
11. Referencias
12. Anexo: Instalación y Despliegue Técnico

3. RESUMEN EJECUTIVO

3.1 Breve descripción del proyecto

El presente Trabajo Final Integrador (TFI) consiste en el diseño, arquitectura y desarrollo de **UniGames**, un Sistema Integral de Entretenimiento Digital implementado como una aplicación web (Single Page Application). La plataforma unifica una amplia variedad de minijuegos lúdicos, de trivia y de aventuras en 2D, centralizando el progreso y la interacción social de los jugadores.

3.2 Objetivos principales

El objetivo general del proyecto es desarrollar un sistema web interactivo que automatice el registro de puntajes, administre perfiles de usuarios y fomente la interactividad a través de sistemas de amistades y duelos asíncronos.
Objetivos específicos:

- Diseñar y normalizar una base de datos relacional para soportar concurrencia.
- Desarrollar una API RESTful escalable bajo Node.js y Express.js.
- Construir un motor de renderizado 2D desde cero (Engine2D) con cálculo de colisiones y delta time.
- Implementar operaciones CRUD avanzadas para el panel de administración global.
- Garantizar la seguridad mediante middleware, encriptación de datos sensibles y saneamiento de entradas.

3.3 Metodología utilizada

El proyecto fue desarrollado utilizando la metodología ágil **Scrum**, organizada en sprints bisemanales. El proceso incluyó relevamiento funcional, diseño de arquitectura, implementación full-stack y pruebas de estrés.

3.4 Resultados esperados y Conclusiones

Se espera obtener una plataforma funcional, capaz de sostener de forma concurrente el registro de usuarios, la interacción social y la carga rápida de minijuegos. El proyecto demuestra competencia en ingeniería de software, arquitectura cliente-servidor y optimización de algoritmos de renderizado gráfico, integrando de manera profunda los conocimientos adquiridos en la carrera.

4. INTRODUCCIÓN

4.1 Contexto y antecedentes

La digitalización del entretenimiento ha llevado a un auge de portales de juegos casuales. Sin embargo, la mayoría de estos portales no retienen el progreso del usuario o dependen de librerías pesadas compiladas a WebAssembly (como Unity).

4.2 Justificación

UniGames responde a la necesidad de construir una plataforma web ligera, segura y altamente social, evitando "cajas negras" y demostrando un dominio absoluto sobre la programación orientada a eventos, el manejo del DOM y las consultas SQL.

4.3 Objetivo General

Desarrollar una plataforma centralizada que ofrezca juegos interactivos con persistencia en la nube, promoviendo la competencia sana mediante tablas de clasificación y duelos directos.

4.4 Objetivos Específicos

- Mantener una SPA rápida y reactiva sin usar frameworks de frontend pesados.
- Crear un sistema de notificaciones en tiempo real.
- Diseñar una interfaz accesible e intuitiva inspirada en interfaces "retro pixel art".

5. ESTADO DEL ARTE

5.1 Contexto tecnológico actual

Las tecnologías web han madurado al punto de igualar el rendimiento de aplicaciones nativas en tareas gráficas básicas gracias a HTML5 `<canvas>` y el motor V8 de JavaScript.

5.2 Sistemas existentes y comparativa

Plataformas como "Miniclip" o "Poki" carecen de una integración social profunda (cuentas y amigos). UniGames retoma la idea de los portales de juegos añadiendo características de redes sociales modernas (perfiles, duelos).

5.3 Enfoques tecnológicos y Modelado

- **Arquitectura:** Monolito modular en Node.js, ideal para asegurar fácil despliegue y mantenimiento. Todo el peso del renderizado (CSR) recae sobre el cliente.
- **Modelado:** PostgreSQL normalizado hasta la Tercera Forma Normal (3FN), garantizando integridad referencial.
- **Seguridad:** Autenticación basada en JSON Web Token (JWT) y Supabase Auth, con protección de rutas mediante middlewares.

5.4 Fundamentación del aporte del proyecto

El mayor aporte técnico es el desarrollo nativo del **Motor Gráfico 2D**, gestionando `requestAnimationFrame`, colisiones AABB, interpolación de animaciones y cálculo de deltatime manualmente.

6. DESCRIPCIÓN DEL PROYECTO

6.1 Problemática y Solución

Ausencia de una plataforma universitaria que demuestre la capacidad de desarrollo full-stack en tiempo real, resuelta mediante el desarrollo de UniGames, un ecosistema web con gestión de usuarios, catálogo de minijuegos, motor 2D y sistema de duelos.

6.2 Historias de Usuario

- **HU1 - Registrar Usuario (Alta):** Como jugador, quiero poder registrarme para guardar mis puntajes.
- **HU2 - Realizar Duelo (Alta):** Como jugador, quiero seleccionar a un amigo y enviarle un desafío.
- **HU3 - Moderación (Alta):** Como administrador, quiero acceder a un panel para banear jugadores conflictivos.

6.3 Reglas de Negocio

- **RN1:** Un nombre de usuario debe ser único en la plataforma.
- **RN2:** No se puede iniciar un duelo si ya existe otro pendiente con el mismo amigo en el mismo juego.
- **RN3:** Los avisos globales solo pueden ser creados por un administrador.

6.4 Modelo Entidad-Relación (E-R)

- **`usuarios`**: `id` (PK), `username`, `email`, `rol`.
- **`amistades`**: `id` (PK), `usuario1_id` (FK), `usuario2_id` (FK), `estado`.
- **`duelos`**: `id` (PK), `retador_id`, `retado_id`, `juego_id`, `estado`.
- **`partidas`**: `id` (PK), `usuario_id`, `juego_id`, `puntos`.

7. METODOLOGÍA Y DESARROLLO

7.1 Enfoque Metodológico

Desarrollo iterativo e incremental basado en **Scrum** y tableros Kanban.

7.2 Diseño del Sistema y Arquitectura

Arquitectura Cliente-Servidor (SPA).

- **Capa de Presentación:** Archivo `index.html` y motor DOM `app2.js`.
- **Capa Lógica:** Controladores de Express.js y middlewares.
- **Capa de Datos:** `database/db.js` abstrae la complejidad de PostgreSQL.

7.3 Pruebas e Integración

Se implementaron simulaciones de peticiones concurrentes para evitar colapsos y se mitigaron vulnerabilidades SQL Injections usando sentencias preparadas.

8. RECURSOS NECESARIOS

- **Recursos Humanos:** Desarrolladores Full-Stack, Analista Funcional, Diseñador de BD.
- **Recursos Materiales:** Entornos Windows, Node.js, GitHub, Supabase, Render/Vercel.

9. PLAN DE TRABAJO (Sprints)

- **Sprint 1:** Diseño E-R, Setup de Supabase, Auth.
- **Sprint 2:** Lógica Motor 2D y Juegos de Trivia.
- **Sprint 3:** Funciones Sociales, Duelos, Panel Admin.
- **Sprint 4:** Refactorización, Pruebas y Despliegue.

10. RESULTADOS ESPERADOS

Demostración concluyente de viabilidad en el desarrollo de plataformas ricas en interactividad sin usar frameworks pesados, aplicando algoritmia 2D, seguridad y bases de datos relacionales requeridos por la Tecnicatura en Programación.

11. REFERENCIAS

1. Mozilla Developer Network (MDN). *Canvas Tutorial*.
2. PostgreSQL Global Development Group. *Documentation*.
3. Express.js Foundation. *Express Framework*.
4. Jones, M. (2015). *JSON Web Token (JWT)*. IETF RFC 7519.

12. ANEXO: INSTALACIÓN Y DESPLIEGUE TÉCNICO

Para los miembros de la mesa evaluadora que deseen auditar el código fuente localmente:

1. Instalar **Node.js (v18+)** y contar con un servidor PostgreSQL local o usar Supabase.
2. Clonar el repositorio: `git clone https://github.com/SuRepositorio/UniGames.git`
3. Instalar dependencias: `npm install`
4. Configurar archivo `.env`:

```
DATABASE_URL=postgresql://usuario:pass@localhost/unigames
SUPABASE_URL=...
SUPABASE_ANON_KEY=...
JWT_SECRET=super_secreto_seguro
```

5. Inicializar la base de datos: `npm run db:reset`
6. Iniciar servidor: `npm run dev` (Disponible en `http://localhost:3000`).